

WEEK – 4, 5

Introduction to Classes, Objects, Constructors, Methods, Parameter Passing, Constructor and Method Overloading

PROGRAM 3 – CLASSES AND OBJECTS (CO1)

1. Title

Farm Animals Java Application using Classes and Objects

2. Objective / Aim

To design and implement a Java application using **classes and objects** to represent farm animals such as cow, pig, and horse, and display information about their place of stay, food, and sounds.

3. Problem Statement

Design a "Farm Animals" Java application containing details of animals such as:

- Cow
- Pig
- Horse

For each animal, store:

- Animal name
- Place where it stays
- Food it eats
- Sound it makes

The program should demonstrate:

- Classes and objects
- Default constructor
- Parameterized constructor
- Methods
- Parameter passing

- Method overloading

4. Concepts Used

- Class and Object
- Default Constructor
- Parameterized Constructor
- Methods
- Parameter Passing
- Method Overloading
- Data Members

5. Algorithm / Logic Description

1. Create a class named `Animal`.
2. Declare data members for name, place, food, and sound.
3. Create a default constructor to assign default values.
4. Create a parameterized constructor to initialize animal details.
5. Create a `displayDetails()` method to display animal information.
6. Create `makeSound(String sound)` to demonstrate parameter passing.
7. Create two overloaded `eat()` methods:
 - o `eat()`
 - o `eat(String food)`
8. Create objects for Cow, Pig, and Horse using the parameterized constructor.
9. Display their details.
10. Pass sounds as arguments to `makeSound()`.
11. Call the overloaded `eat()` methods.
12. Create another object using the default constructor.
13. Display the default values.

6. Source Code

```
package recordprograms;

import java.util.Scanner;

// Farm Animals Java Application

class Animal {

    // Data members
    String name;
    String place;
    String food;
    String sound;

    // Default constructor
    Animal() {
```

```

        name = "Unknown";
        place = "Farm";
        food = "Unknown";
        sound = "Unknown";
    }

    // Parameterized constructor
    Animal(String name, String place, String food, String sound) {
        this.name = name;
        this.place = place;
        this.food = food;
        this.sound = sound;
    }

    // Method to display animal details
    void displayDetails() {
        System.out.println("Animal : " + name);
        System.out.println("Stays : " + place);
        System.out.println("Eats : " + food);
        System.out.println("Sound : " + sound);
        System.out.println();
    }

    // Method with parameter - parameter passing
    void makeSound(String sound) {
        System.out.println(name + " makes sound: " + sound);
    }

    // Overloaded method
    void eat() {
        System.out.println(name + " is eating.");
    }

    // Overloaded method with parameter
    void eat(String food) {
        System.out.println(name + " eats " + food + ".");
    }
}

public class FarmAnimals {

    public static void main(String[] args) {

        // Creating objects using parameterized constructor
        Animal cow = new Animal(
            "Cow",
            "Cowshed",
            "Grass",
            "Moo"
        );

        Animal pig = new Animal(
            "Pig",
            "Pigsty",
            "Grains",
            "Oink"
        );
    }
}

```

```

        Animal horse = new Animal(
            "Horse",
            "Stable",
            "Hay",
            "Neigh"
        );

        // Display details
        System.out.println("===== FARM ANIMALS =====");

        cow.displayDetails();
        pig.displayDetails();
        horse.displayDetails();

        // Parameter passing
        System.out.println("===== ANIMAL SOUNDS =====");

        cow.makeSound("Moo");
        pig.makeSound("Oink");
        horse.makeSound("Neigh");

        // Method overloading
        System.out.println("\n===== EATING =====");

        cow.eat();
        pig.eat("Grains");
        horse.eat("Hay");

        // Creating object using default constructor
        Animal animal = new Animal();

        System.out.println("\n===== DEFAULT CONSTRUCTOR =====");

        animal.displayDetails();
    }
}

```

7. Sample Output

```

===== FARM ANIMALS =====
Animal : Cow
Stays  : Cowshed
Eats   : Grass
Sound  : Moo

Animal : Pig
Stays  : Pigsty
Eats   : Grains
Sound  : Oink

Animal : Horse
Stays  : Stable
Eats   : Hay
Sound  : Neigh

===== ANIMAL SOUNDS =====

```

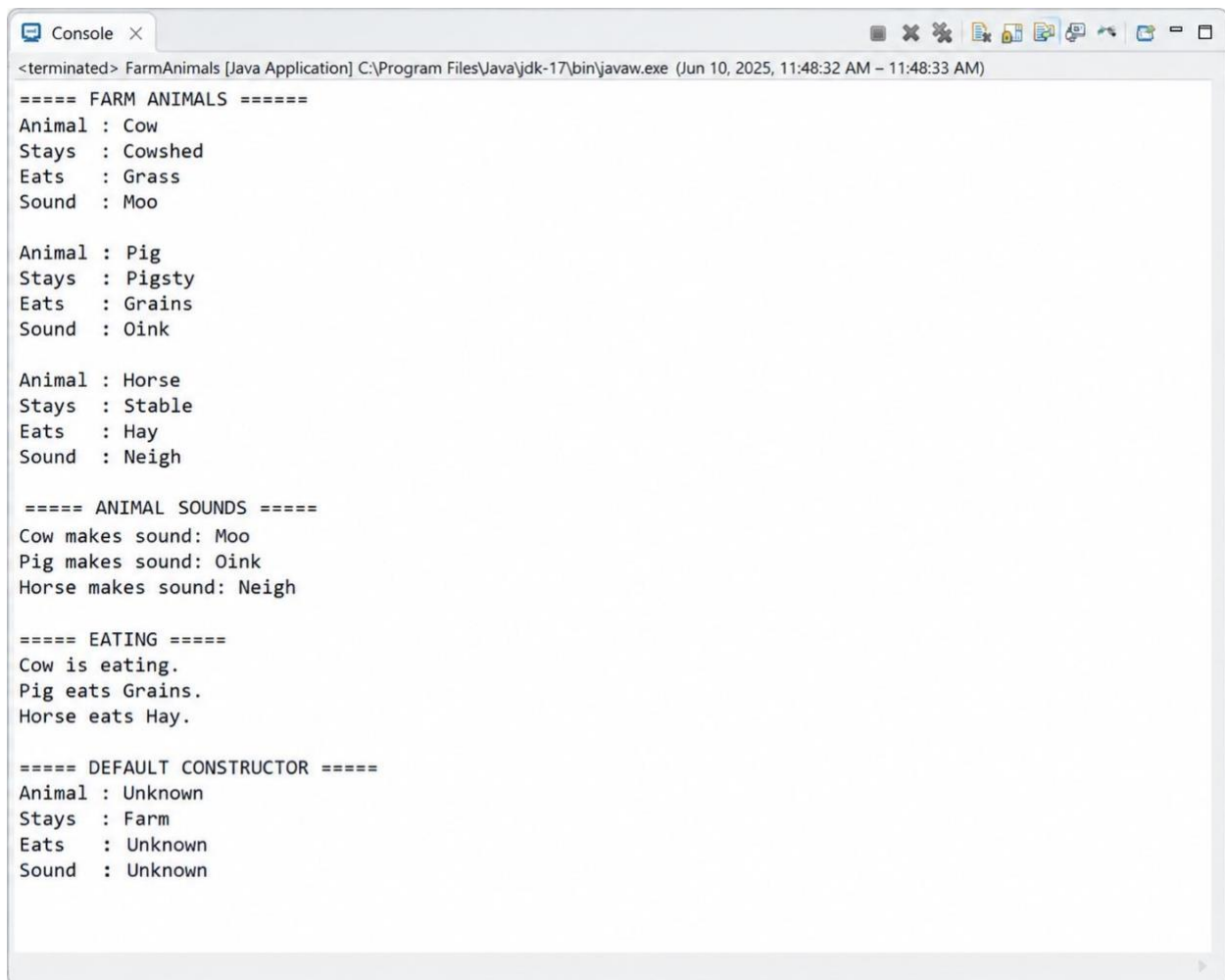
```
Cow makes sound: Moo
Pig makes sound: Oink
Horse makes sound: Neigh
```

```
===== EATING =====
```

```
Cow is eating.
Pig eats Grains.
Horse eats Hay.
```

```
===== DEFAULT CONSTRUCTOR =====
```

```
Animal : Unknown
Stays   : Farm
Eats    : Unknown
Sound   : Unknown
```

A screenshot of a Java console window titled 'Console'. The window shows the output of a Java application named 'FarmAnimals'. The output is organized into several sections: 'FARM ANIMALS' which lists details for Cow, Pig, and Horse; 'ANIMAL SOUNDS' which shows the sound each animal makes; 'EATING' which shows what each animal is eating; and 'DEFAULT CONSTRUCTOR' which shows the default state of an animal object. The console window has a standard Windows-style title bar and a toolbar with various icons for file operations and window management.

```
<terminated> FarmAnimals [Java Application] C:\Program Files\Java\jdk-17\bin\javaw.exe (Jun 10, 2025, 11:48:32 AM - 11:48:33 AM)

===== FARM ANIMALS =====
Animal : Cow
Stays   : Cowshed
Eats    : Grass
Sound   : Moo

Animal : Pig
Stays   : Pigsty
Eats    : Grains
Sound   : Oink

Animal : Horse
Stays   : Stable
Eats    : Hay
Sound   : Neigh

===== ANIMAL SOUNDS =====
Cow makes sound: Moo
Pig makes sound: Oink
Horse makes sound: Neigh

===== EATING =====
Cow is eating.
Pig eats Grains.
Horse eats Hay.

===== DEFAULT CONSTRUCTOR =====
Animal : Unknown
Stays   : Farm
Eats    : Unknown
Sound   : Unknown
```

8. Result

The Farm Animals application was successfully implemented using **classes, objects, constructors, methods, parameter passing, and method overloading** in Java.

PROGRAM 4 – CLASSES AND OBJECTS (CO1)

1. Title

Order Management System using Classes and Inheritance

2. Objective / Aim

To design and implement a Java program based on a class diagram to demonstrate **classes, objects, constructors, inheritance, and methods** using a customer and different types of orders.

3. Problem Statement

Develop an Order Management System containing:

- Customer
- Order
- SpecialOrder
- NormalOrder

A customer should be able to send and receive orders. An order should support confirmation and closing. Special and normal orders should have their own dispatch operations.

4. Concepts Used

- Classes and Objects
- Constructors
- Methods
- Inheritance
- `super` keyword
- Object creation
- Method implementation

5. Algorithm / Logic Description

1. Create a `Customer` class with name and location.
2. Create methods to send and receive orders.
3. Create an `Order` parent class with date and order number.
4. Create `confirm()` and `close()` methods.
5. Create `SpecialOrder` as a child class of `Order`.
6. Add a `dispatch()` method for special orders.
7. Create `NormalOrder` as another child class of `Order`.
8. Add `dispatch()` and `receive()` methods for normal orders.

9. Create objects for the customer, special order, and normal order.
10. Perform customer and order operations.
11. Display the order delivery status.

6. Source Code

```
package recordprograms;

import java.util.*;

// Customer class
class Customer {
    String name;
    String location;

    Customer(String name, String location) {
        this.name = name;
        this.location = location;
    }

    void sendOrder() {
        System.out.println(name + " has sent the order.");
    }

    void receiveOrder() {
        System.out.println(name + " has received the order.");
    }
}

// Order class - Parent class
class Order {
    String date;
    String number;

    Order(String date, String number) {
        this.date = date;
        this.number = number;
    }

    void confirm() {
        System.out.println("Order " + number + " is confirmed.");
    }

    void close() {
        System.out.println("Order " + number + " is closed.");
    }
}

// SpecialOrder inherits Order
class SpecialOrder extends Order {

    SpecialOrder(String date, String number) {
        super(date, number);
    }
}
```

```

        void dispatch() {
            System.out.println("Special order " + number + " is dispatched.");
        }
    }

// NormalOrder inherits Order
class NormalOrder extends Order {

    NormalOrder(String date, String number) {
        super(date, number);
    }

    void dispatch() {
        System.out.println("Normal order " + number + " is dispatched.");
    }

    void receive() {
        System.out.println("Normal order " + number + " is received.");
    }
}

// Main class
public class OrderManagement {

    public static void main(String[] args) {

        Customer customer =
            new Customer("Rahul", "Chennai");

        SpecialOrder specialOrder =
            new SpecialOrder("26-08-2026", "S001");

        NormalOrder normalOrder =
            new NormalOrder("26-08-2026", "N001");

        System.out.println("===== CUSTOMER =====");

        System.out.println("Name      : " + customer.name);
        System.out.println("Location : " + customer.location);

        customer.sendOrder();

        System.out.println("\n===== SPECIAL ORDER =====");

        System.out.println("Date    : " + specialOrder.date);
        System.out.println("Number  : " + specialOrder.number);

        specialOrder.confirm();
        specialOrder.dispatch();
        specialOrder.close();

        System.out.println("\n===== NORMAL ORDER =====");

        System.out.println("Date    : " + normalOrder.date);
        System.out.println("Number  : " + normalOrder.number);

        normalOrder.confirm();
    }
}

```

```

        normalOrder.dispatch();
        normalOrder.receive();
        normalOrder.close();

        System.out.println("\n===== ORDER DELIVERY =====");

        customer.receiveOrder();
    }
}

```

7. Sample Output

```

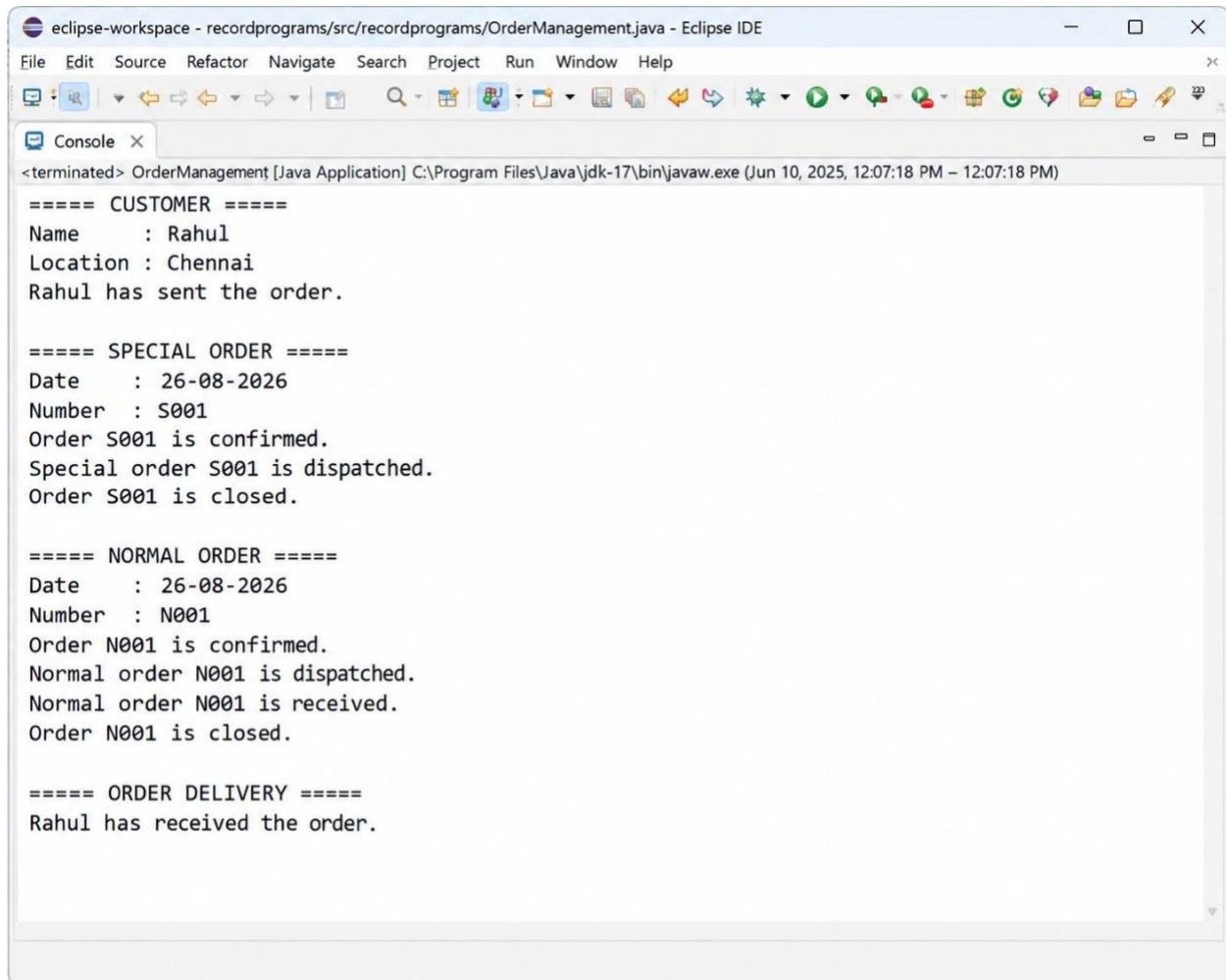
===== CUSTOMER =====
Name      : Rahul
Location  : Chennai
Rahul has sent the order.

===== SPECIAL ORDER =====
Date      : 26-08-2026
Number    : S001
Order S001 is confirmed.
Special order S001 is dispatched.
Order S001 is closed.

===== NORMAL ORDER =====
Date      : 26-08-2026
Number    : N001
Order N001 is confirmed.
Normal order N001 is dispatched.
Normal order N001 is received.
Order N001 is closed.

===== ORDER DELIVERY =====
Rahul has received the order.

```



The screenshot shows the Eclipse IDE interface with the 'Console' window open. The console displays the output of a Java application named 'OrderManagement'. The output is organized into four sections, each preceded by a separator line of five equals signs. The first section, 'CUSTOMER', shows a customer named 'Rahul' from 'Chennai' who has sent an order. The second section, 'SPECIAL ORDER', shows a date of '26-08-2026', order number 'S001', and a confirmation that the special order is dispatched and closed. The third section, 'NORMAL ORDER', shows the same date, order number 'N001', and a confirmation that the normal order is dispatched, received, and closed. The fourth section, 'ORDER DELIVERY', shows that 'Rahul has received the order'.

```
eclipse-workspace - recordprograms/src/recordprograms/OrderManagement.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Console X
<terminated> OrderManagement [Java Application] C:\Program Files\Java\jdk-17\bin\javaw.exe (Jun 10, 2025, 12:07:18 PM - 12:07:18 PM)

===== CUSTOMER =====
Name      : Rahul
Location  : Chennai
Rahul has sent the order.

===== SPECIAL ORDER =====
Date       : 26-08-2026
Number     : S001
Order S001 is confirmed.
Special order S001 is dispatched.
Order S001 is closed.

===== NORMAL ORDER =====
Date       : 26-08-2026
Number     : N001
Order N001 is confirmed.
Normal order N001 is dispatched.
Normal order N001 is received.
Order N001 is closed.

===== ORDER DELIVERY =====
Rahul has received the order.
```

8. Result

The Order Management System was successfully implemented using **classes, objects, constructors, inheritance, and methods** in Java.

PROGRAM 5 – CONSTRUCTOR OVERLOADING (CO1)

1. Title

Employee Data Management using Constructor Overloading

2. Objective / Aim

To design and implement a Java program to maintain employee information using **constructor overloading**, where different constructors initialize employee objects with different numbers of parameters.

3. Problem Statement

An organization maintains employee information according to the employee's cadre.

The employee details include:

- Name
- ID
- Designation
- Salary
- Promotion Status

Implement the system using **constructor overloading**.

4. Concepts Used

- Class and Object
- Constructors
- Constructor Overloading
- `this` keyword
- Methods
- Parameter Passing

5. Algorithm / Logic Description

1. Create an `Employee` class.
2. Declare name, ID, designation, salary, and promotion status.
3. Create a default constructor.
4. Create a constructor with name and ID.
5. Create a constructor with name, ID, and designation.
6. Create a constructor with all five parameters.
7. Initialize the employee details using the appropriate constructor.
8. Create a `display()` method to display employee details.
9. Create four employee objects using different constructors.
10. Display the details of all employees.

6. Constructor Overloading

The program contains four constructors:

```
Employee()  
Employee(String name, int id)  
Employee(String name, int id, String designation)  
Employee(String name, int id, String designation,  
          double salary, String promotionStatus)
```

All constructors have the **same name** but different parameter lists. This is called **constructor overloading**.

7. Source Code

```
package recordprograms;  
  
import java.util.Scanner;  
  
class Employee {  
  
    String name;  
    int id;  
    String designation;  
    double salary;  
    String promotionStatus;  
  
    // Constructor 1: No arguments  
    Employee() {  
        name = "Unknown";  
        id = 0;  
        designation = "Not Assigned";  
        salary = 0.0;  
        promotionStatus = "Not Eligible";  
    }  
  
    // Constructor 2: Name and ID  
    Employee(String name, int id) {  
        this.name = name;  
        this.id = id;  
        designation = "Employee";  
        salary = 20000;  
        promotionStatus = "Pending";  
    }  
  
    // Constructor 3: Name, ID and Designation  
    Employee(String name, int id, String designation) {  
        this.name = name;  
        this.id = id;  
        this.designation = designation;  
        salary = 30000;  
        promotionStatus = "Pending";  
    }  
}
```

```

// Constructor 4: All parameters
Employee(String name, int id, String designation,
         double salary, String promotionStatus) {
    this.name = name;
    this.id = id;
    this.designation = designation;
    this.salary = salary;
    this.promotionStatus = promotionStatus;
}

// Method to display employee details
void display() {
    System.out.println("Name           : " + name);
    System.out.println("ID           : " + id);
    System.out.println("Designation       : " + designation);
    System.out.println("Salary           : Rs." + salary);
    System.out.println("Promotion Status : " + promotionStatus);
    System.out.println();
}
}

// Main class
public class EmployeeDemo {

    public static void main(String[] args) {

        // Object using default constructor
        Employee e1 = new Employee();

        // Object using constructor with 2 parameters
        Employee e2 = new Employee("Arun", 101);

        // Object using constructor with 3 parameters
        Employee e3 = new Employee(
            "Priya", 102, "Manager");

        // Object using constructor with 5 parameters
        Employee e4 = new Employee(
            "Rahul", 103, "Senior Manager",
            75000, "Promoted");

        System.out.println("==== EMPLOYEE DETAILS =====");

        e1.display();
        e2.display();
        e3.display();
        e4.display();
    }
}

```

8. Sample Output

```
==== EMPLOYEE DETAILS =====
```

```

Name           : Unknown
ID            : 0
Designation    : Not Assigned

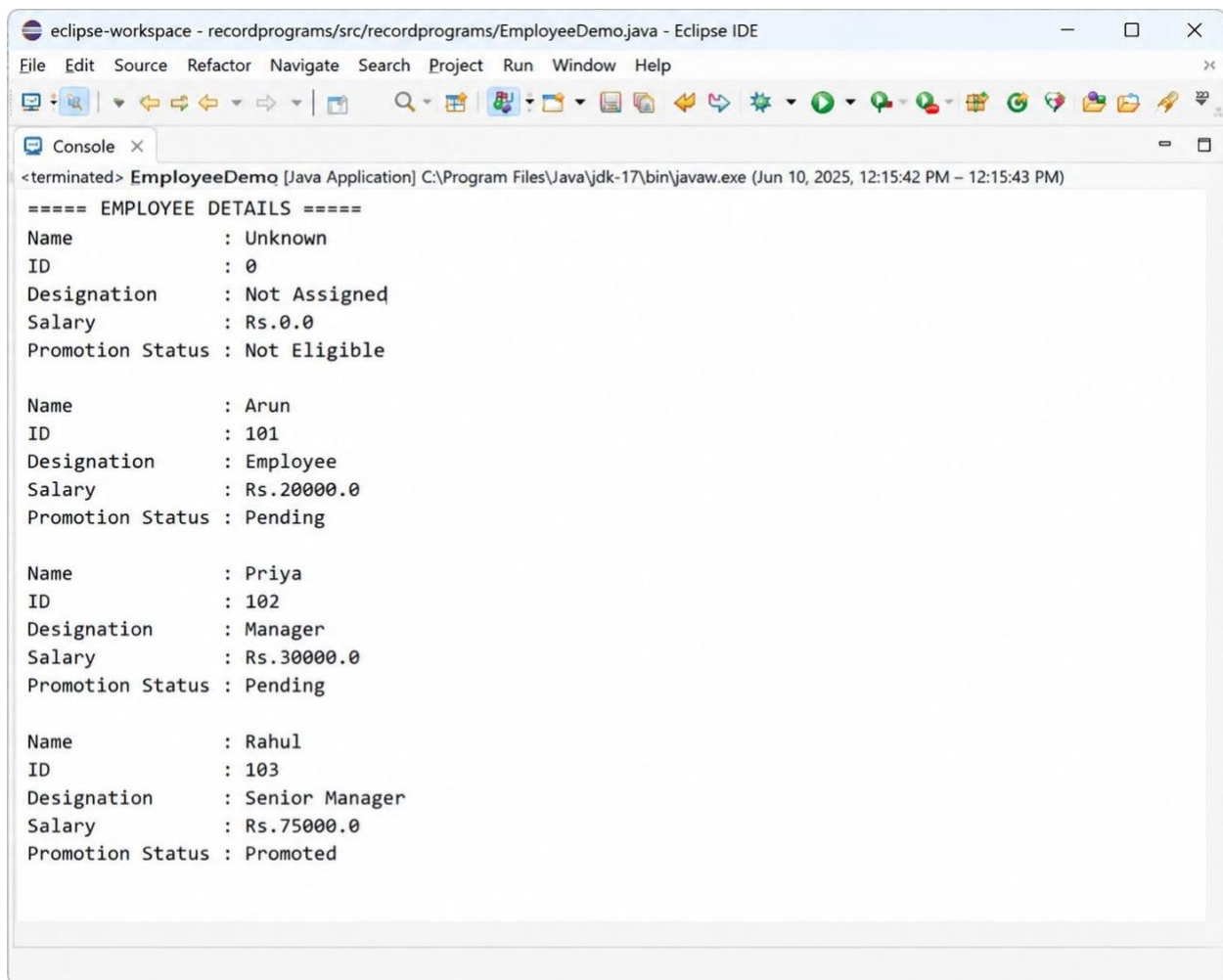
```

Salary : Rs.0.0
Promotion Status : Not Eligible

Name : Arun
ID : 101
Designation : Employee
Salary : Rs.20000.0
Promotion Status : Pending

Name : Priya
ID : 102
Designation : Manager
Salary : Rs.30000.0
Promotion Status : Pending

Name : Rahul
ID : 103
Designation : Senior Manager
Salary : Rs.75000.0
Promotion Status : Promoted



The screenshot shows the Eclipse IDE interface with the console window open. The title bar reads 'eclipse-workspace - recordprograms/src/recordprograms/EmployeeDemo.java - Eclipse IDE'. The menu bar includes File, Edit, Source, Refactor, Navigate, Search, Project, Run, Window, and Help. The toolbar contains various icons for file operations and development tools. The console window title is 'Console X' and the output text is as follows:

```
<terminated> EmployeeDemo [Java Application] C:\Program Files\Java\jdk-17\bin\javaw.exe (Jun 10, 2025, 12:15:42 PM – 12:15:43 PM)

===== EMPLOYEE DETAILS =====
Name      : Unknown
ID        : 0
Designation : Not Assigned
Salary    : Rs.0.0
Promotion Status : Not Eligible

Name      : Arun
ID        : 101
Designation : Employee
Salary    : Rs.20000.0
Promotion Status : Pending

Name      : Priya
ID        : 102
Designation : Manager
Salary    : Rs.30000.0
Promotion Status : Pending

Name      : Rahul
ID        : 103
Designation : Senior Manager
Salary    : Rs.75000.0
Promotion Status : Promoted
```

9. Result

The Employee Data Management program was successfully implemented using **classes, objects, constructors, and constructor overloading**. Different constructors were used to initialize employee objects with different sets of information.

CONCLUSION

The programs successfully demonstrate the fundamental concepts of Java Object-Oriented Programming, including **classes, objects, constructors, parameter passing, constructor overloading, method overloading, and inheritance**.